In [1]:
```
import dash
from dash import dcc, html
from dash.dependencies import Input, Output
import pandas as pd
import plotly.express as px
import dash_bootstrap_components as dbc
```

In [2]:
```
# Load the dataset
file_path = 'company_distribution_data.csv'
data = pd.read_csv(file_path)
data
```

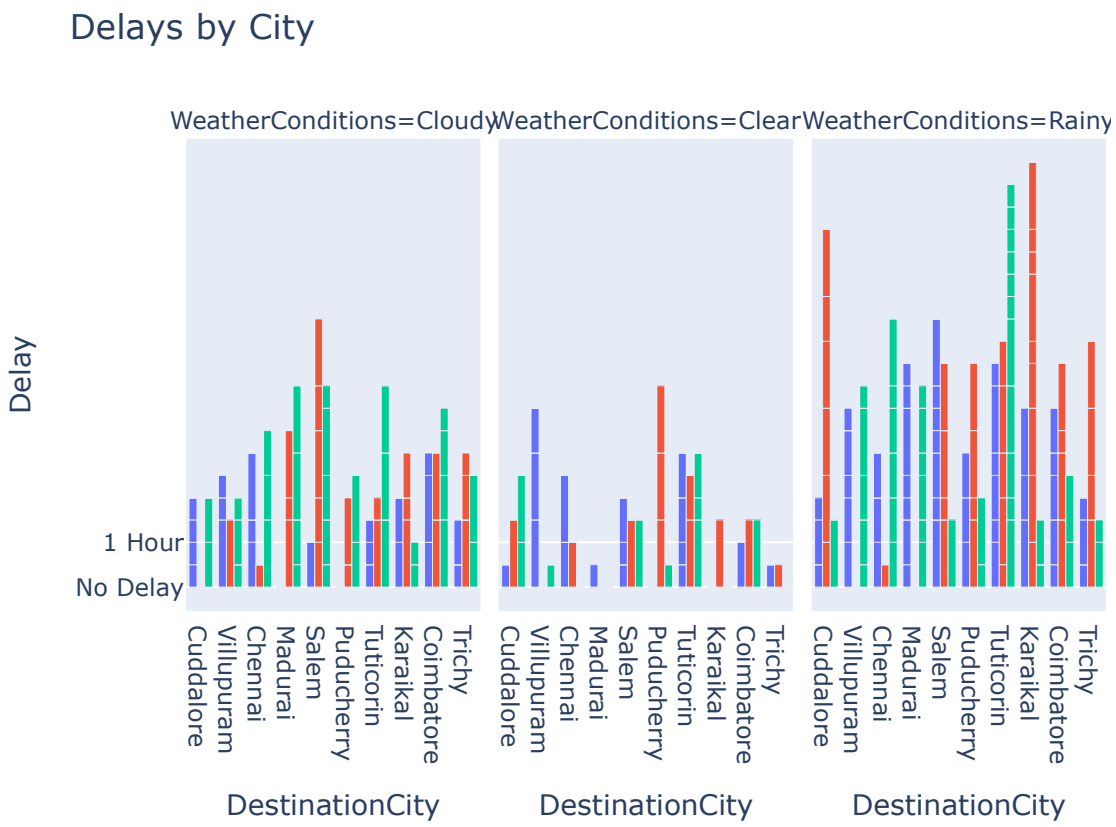
Out[2]:

	DispatchID	DispatchDate	ProductID	ProductName	Quantity	Destination	DestinationCity
0	170	13-07-2024	1005	Digestion Biscuits	367	RetailerB	Cuddalore
1	248	13-07-2024	1004	Cream Biscuit	662	RetailerC	Salem
2	318	13-07-2024	1001	Chips	282	RetailerB	Villupuram
3	33	12-07-2024	1004	Cream Biscuit	892	RetailerA	Chennai
4	120	12-07-2024	1005	Digestion Biscuits	830	RetailerD	Madurai
...	...	...	...	...	...	...	...
841	772	17-07-2023	1004	Cream Biscuit	861	RetailerA	Coimbatore
842	292	16-07-2023	1003	Coconut Biscuit	385	RetailerD	Tuticorin
843	433	15-07-2023	1006	Sugar Biscuit	293	RetailerD	Puducherry
844	509	15-07-2023	1005	Digestion Biscuits	892	RetailerC	Villupuram
845	752	15-07-2023	1006	Sugar Biscuit	771	RetailerC	Tuticorin

846 rows × 16 columns

In [3]:
```
# Load the dataset
file_path = 'company_distribution_data.csv'
data = pd.read_csv(file_path)

# Initialize the Dash app
app = dash.Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])

# Layout of the dashboard
app.layout = dbc.Container([
    html.H1("Real-Time Delay Dashboard"),

    dbc.Row([
        dbc.Col([
            dcc.Graph(id='delay-conditions-chart')
        ], width=12),
    ]),

    dcc.Interval(
        id='interval-component',
        interval=1*60*1000,  # in milliseconds (1 minute)
        n_intervals=0
    )
])

# Callback to update the chart with new data
@app.callback(
    Output('delay-conditions-chart', 'figure'),
    [Input('interval-component', 'n_intervals')]
)
def update_chart(n):
    # Reload the data (assuming the file is updated periodically)
    updated_data = pd.read_csv(file_path)

    # Filter only the needed columns
    relevant_data = updated_data[['DestinationCity', 'Delay', 'WeatherConditio

    # Create a grouped bar chart
    fig = px.bar(
        relevant_data,
        x='DestinationCity',
        y='Delay',
        color='TrafficConditions',
        barmode='group',
        facet_col='WeatherConditions',
        title='Delays by City',
        labels={'Delay': 'Delay'}
    )

    return fig

# Run the app
if __name__ == '__main__':
    app.run_server(debug=True,port=8055)
```

Real-Time Delay Dashboard



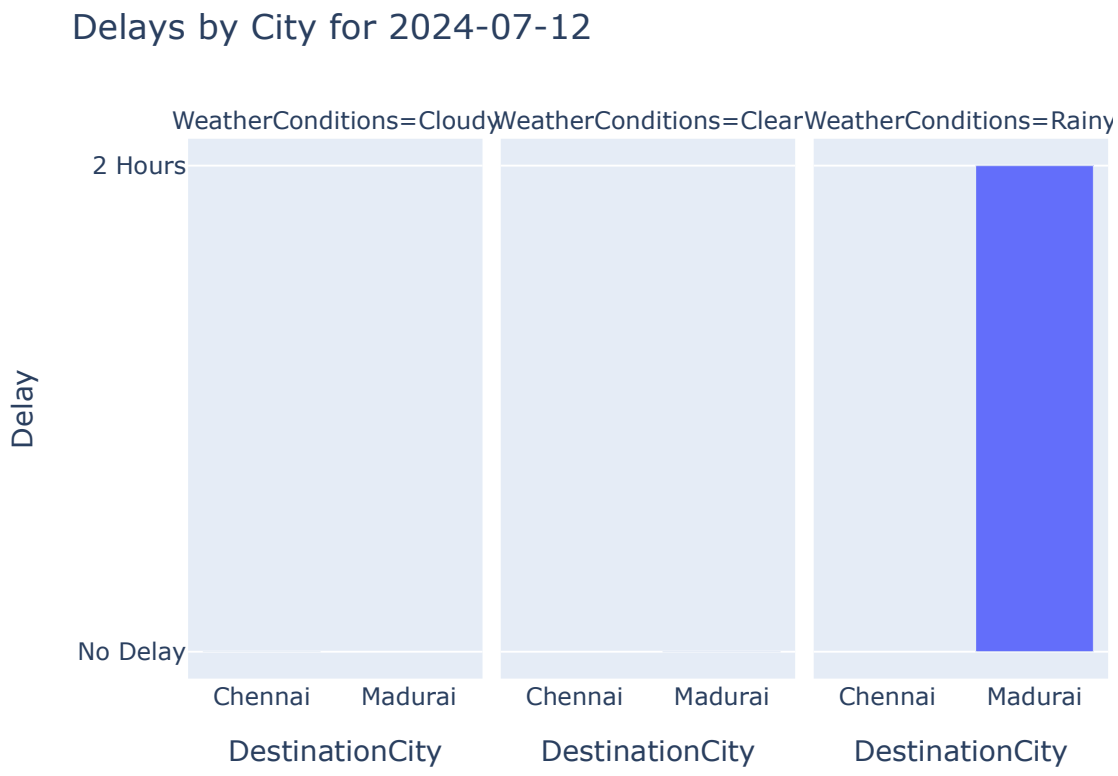
REAL TIME DASHBOARD FOR SPECIFIC DATE

In [4]:
```
# Convert the 'Date' column to datetime
data['DispatchDate'] = pd.to_datetime(data['DispatchDate'])

# Initialize the Dash app
app = dash.Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])

# Layout of the dashboard
app.layout = dbc.Container([
    html.H1("Real-Time Delay Dashboard"),

    dbc.Row([
        dbc.Col([
            dcc.Graph(id='delay-conditions-chart')
        ], width=12),
    ]),

    # Add the DatePickerSingle component for date selection
    dbc.Row([
        dbc.Col([
            dcc.DatePickerSingle(
                id='date-picker',
                min_date_allowed=data['DispatchDate'].min(),
                max_date_allowed=data['DispatchDate'].max(),
                initial_visible_month=data['DispatchDate'].min().date(),  # Se
                date='2024-07-12' # Set default date
            )
        ], width=6)
    ]),

    dcc.Interval(
        id='interval-component',
        interval=1*60*1000,  # in milliseconds (1 minute)
        n_intervals=0
    )
])

# Callback to update the chart based on selected date
@app.callback(
    Output('delay-conditions-chart', 'figure'),
    [Input('interval-component', 'n_intervals'),
     Input('date-picker', 'date')]
)
def update_chart(n, selected_date):
    # Reload the data (assuming the file is updated periodically)
    updated_data = pd.read_csv(file_path)

    # Convert the 'Date' column to datetime if not already
    updated_data['DispatchDate'] = pd.to_datetime(updated_data['DispatchDate']

    # Filter data for the selected date
    filtered_data = updated_data[updated_data['DispatchDate'] == pd.to_datetim

    # Check if there is data for the selected date
    if filtered_data.empty:
        # Return an empty plot with a message
```

```
        fig = px.bar(title=f"No data available for {selected_date}")
    else:
        # Filter only the needed columns
        relevant_data = filtered_data[['DestinationCity', 'Delay', 'WeatherCon

        # Create a grouped bar chart
        fig = px.bar(
            relevant_data,
            x='DestinationCity',
            y='Delay',
            color='TrafficConditions',
            barmode='group',
            facet_col='WeatherConditions',
            title=f'Delays by City for {selected_date}',
            labels={'Delay': 'Delay'}
        )

    return fig

# Run the app
if __name__ == '__main__':
    app.run_server(debug=True, port=8056)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_2908\1336636688.py:2: UserWarning:

Parsing dates in %d-%m-%Y format when dayfirst=False (the default) was specif
ied. Pass `dayfirst=True` or specify a format to silence this warning.
```

Real-Time Delay Dashboard



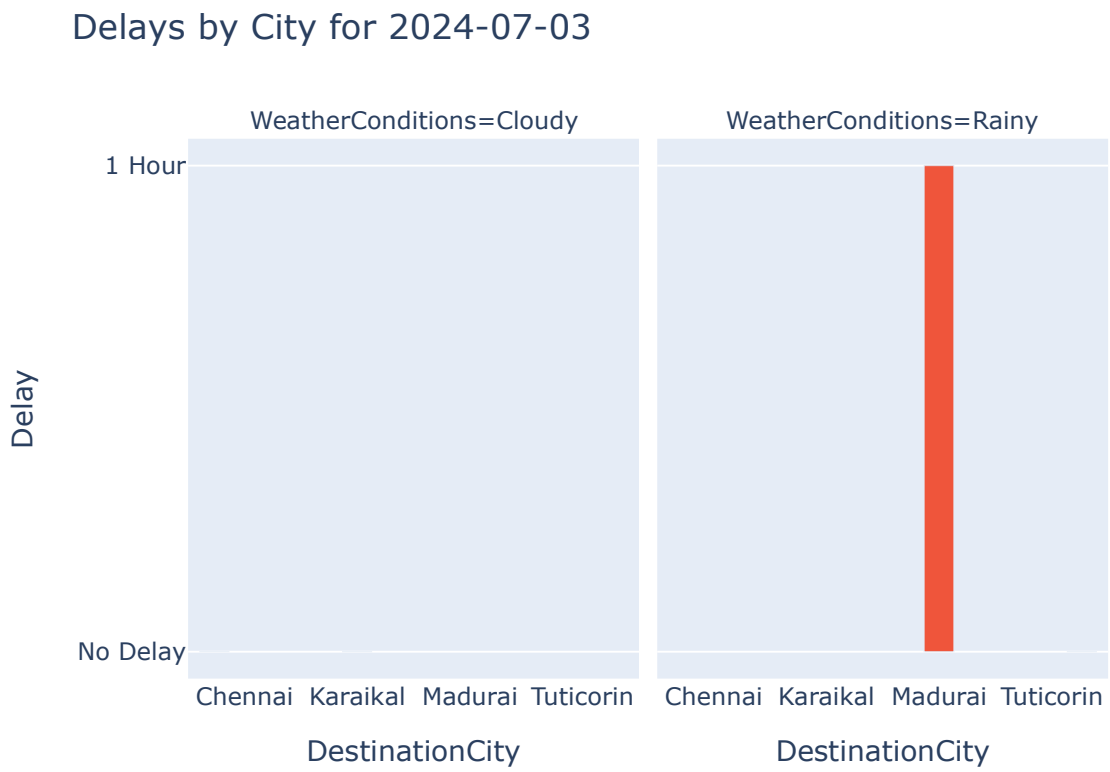
07/12/2024

```
C:\Users\HP\AppData\Local\Temp\ipykernel_2908\1336636688.py:48: UserWarning:

Parsing dates in %d-%m-%Y format when dayfirst=False (the default) was specif
ied. Pass `dayfirst=True` or specify a format to silence this warning.
```

In [5]:
```
# Convert the 'Date' column to datetime
data['DispatchDate'] = pd.to_datetime(data['DispatchDate'])

# Initialize the Dash app
app = dash.Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])

# Layout of the dashboard
app.layout = dbc.Container([
    html.H1("Real-Time Delay Dashboard"),

    dbc.Row([
        dbc.Col([
            dcc.Graph(id='delay-conditions-chart')
        ], width=12),
    ]),

    # Add the DatePickerSingle component for date selection
    dbc.Row([
        dbc.Col([
            dcc.DatePickerSingle(
                id='date-picker',
                min_date_allowed=data['DispatchDate'].min(),
                max_date_allowed=data['DispatchDate'].max(),
                initial_visible_month=data['DispatchDate'].min().date(),  # Se
                date='2024-07-03' # Set default date
            )
        ], width=6)
    ]),

    dcc.Interval(
        id='interval-component',
        interval=1*60*1000,  # in milliseconds (1 minute)
        n_intervals=0
    )
])

# Callback to update the chart based on selected date
@app.callback(
    Output('delay-conditions-chart', 'figure'),
    [Input('interval-component', 'n_intervals'),
     Input('date-picker', 'date')]
)
def update_chart(n, selected_date):
    # Reload the data (assuming the file is updated periodically)
    updated_data = pd.read_csv(file_path)

    # Convert the 'Date' column to datetime if not already
    updated_data['DispatchDate'] = pd.to_datetime(updated_data['DispatchDate']

    # Filter data for the selected date
    filtered_data = updated_data[updated_data['DispatchDate'] == pd.to_datetim

    # Check if there is data for the selected date
    if filtered_data.empty:
        # Return an empty plot with a message
```

```
        fig = px.bar(title=f"No data available for {selected_date}")
    else:
        # Filter only the needed columns
        relevant_data = filtered_data[['DestinationCity', 'Delay', 'WeatherCon

        # Create a grouped bar chart
        fig = px.bar(
            relevant_data,
            x='DestinationCity',
            y='Delay',
            color='TrafficConditions',
            barmode='group',
            facet_col='WeatherConditions',
            title=f'Delays by City for {selected_date}',
            labels={'Delay': 'Delay'}
        )

    return fig

# Run the app
if __name__ == '__main__':
    app.run_server(debug=True, port=8056)
```

Real-Time Delay Dashboard



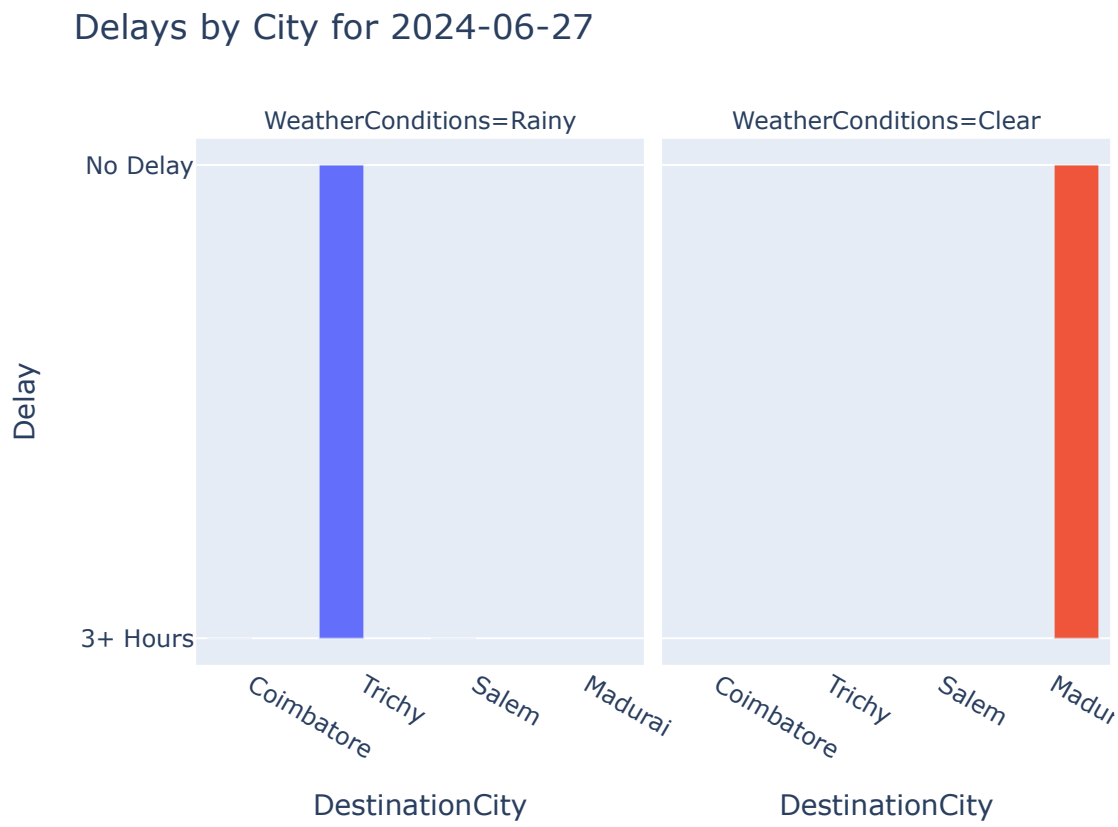
07/03/2024

```
C:\Users\HP\AppData\Local\Temp\ipykernel_2908\2856163802.py:48: UserWarning:

Parsing dates in %d-%m-%Y format when dayfirst=False (the default) was specif
ied. Pass `dayfirst=True` or specify a format to silence this warning.
```

In [6]:
```
# Convert the 'Date' column to datetime
data['DispatchDate'] = pd.to_datetime(data['DispatchDate'])

# Initialize the Dash app
app = dash.Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])

# Layout of the dashboard
app.layout = dbc.Container([
    html.H1("Real-Time Delay Dashboard"),

    dbc.Row([
        dbc.Col([
            dcc.Graph(id='delay-conditions-chart')
        ], width=12),
    ]),

    # Add the DatePickerSingle component for date selection
    dbc.Row([
        dbc.Col([
            dcc.DatePickerSingle(
                id='date-picker',
                min_date_allowed=data['DispatchDate'].min(),
                max_date_allowed=data['DispatchDate'].max(),
                initial_visible_month=data['DispatchDate'].min().date(),  # Se
                date='2024-06-27' # Set default date
            )
        ], width=6)
    ]),

    dcc.Interval(
        id='interval-component',
        interval=1*60*1000,  # in milliseconds (1 minute)
        n_intervals=0
    )
])

# Callback to update the chart based on selected date
@app.callback(
    Output('delay-conditions-chart', 'figure'),
    [Input('interval-component', 'n_intervals'),
     Input('date-picker', 'date')]
)
def update_chart(n, selected_date):
    # Reload the data (assuming the file is updated periodically)
    updated_data = pd.read_csv(file_path)

    # Convert the 'Date' column to datetime if not already
    updated_data['DispatchDate'] = pd.to_datetime(updated_data['DispatchDate']

    # Filter data for the selected date
    filtered_data = updated_data[updated_data['DispatchDate'] == pd.to_datetim

    # Check if there is data for the selected date
    if filtered_data.empty:
        # Return an empty plot with a message
```

```
        fig = px.bar(title=f"No data available for {selected_date}")
    else:
        # Filter only the needed columns
        relevant_data = filtered_data[['DestinationCity', 'Delay', 'WeatherCon

        # Create a grouped bar chart
        fig = px.bar(
            relevant_data,
            x='DestinationCity',
            y='Delay',
            color='TrafficConditions',
            barmode='group',
            facet_col='WeatherConditions',
            title=f'Delays by City for {selected_date}',
            labels={'Delay': 'Delay'}
        )

    return fig

# Run the app
if __name__ == '__main__':
    app.run_server(debug=True,port=8058)
```

Real-Time Delay Dashboard




```
C:\Users\HP\AppData\Local\Temp\ipykernel_2908\1196621686.py:48: UserWarning:

Parsing dates in %d-%m-%Y format when dayfirst=False (the default) was specif
ied. Pass `dayfirst=True` or specify a format to silence this warning.
```

In [ ]: